

ESP32 Wetter Station Teil 2

Anthony Timmel

13. Mai 2026

Inhaltsverzeichnis

1 Grundlagen	4
1.1 Begriffe des MQTT-Protokolls	4
1.1.1 MQTT-Broker	4
1.1.2 MQTT-Client	4
1.1.3 MQTT-Topic und dessen Grundregeln	4
1.1.4 Topic Wildcard einstufig und mehrstufig	4
1.1.5 Stufenmodell Quality of Service (QoS)	5
1.1.6 Last Will Testament	6
1.2 Verbindungsmanagement zwischen Client und Server	7
1.2.1 Retained Messages	7
1.2.2 Persistent Sessions	7
1.2.3 Keep Alive	7
1.2.4 Web Sockets Transport	7
1.3 Pub/Sub Messaging	7
1.4 MQTT-Client	8
2 Internetverbindung für den ESP32	9
2.1 Planung	9
2.2 Programmcode	9
3 MQTT-Client	11
3.1 Planung	11
3.2 Programmcode	11
4 Verarbeitung der Daten in NODE-RED	12
4.1 MQTT-Client innerhalb von NODE-RED	12
4.2 Temperatur	14
4.2.1 Vorbereitung der Daten für das UI	14
4.2.2 UI Template für aktuelle Daten	14
4.2.3 UI Template für historie Daten	15
4.3 Luftfeuchtigkeit	15
4.3.1 Vorbereitung der Daten für das UI	15
4.3.2 UI Template für aktuelle Daten	15
4.3.3 UI Template für historie Daten	15
4.4 Bewegung	15
4.4.1 Vorbereitung der Daten für das UI	15
4.4.2 UI Template für aktuelle Daten	15
4.4.3 UI Template für historie Daten	15
4.5 Helligkeit	15
4.5.1 Vorbereitung der Daten für das UI	15
4.5.2 UI Template für aktuelle Daten	15

4.5.3	UI Template für historie Daten	15
-------	--	----

Im Rahmen des ersten Teils dieser Dokumentation setzen wir uns intensiv mit der initialen Aufgabenstellung aus dem Unterricht des Lernfeldes 7 (LF7) auseinander. Auf den nachfolgenden Seiten wird Ihnen das Projekt *Wetterstation ESP32* in seiner zweiten Phase detailliert und in schriftlicher Form präsentiert, um einen fundierten Überblick über die Zielsetzung und den Aufbau zu geben.

Diese Dokumentation ist der zweite Teil der Dokumentationsreihe, über den *ESP32* als Wetterstation.

Kapitel 1

Grundlagen

Innerhalb des zweiten Teils, kommen neue softwarebasierte Technologien zum Einsatz. Damit, bei der Referenzierung auf diese Begriffe & bei dem Ablauf der Beschreibung des Programmcodes keine Komplikationen in dem Verständniss des Ablaufs auftreten, werden die Begrifflichkeiten, im Umfang der zweiten Aufgabenstellung erklärt. Für eine detailliertere Beschreibung der einzelnen Begriffe, empfiehlt sich das Lesen der Beiträge auf [HiveMQ](#)

1.1 Begriffe des MQTT-Protokolls

In den folgenden Sektionen, werden Ihnen folgende Begriffe erklärt. Dies geschieht innerhalb des Rahmens von *Skript Teil 3*.

1.1.1 MQTT-Broker

Der MQTT-Broker ist vergleichbar mit der Poststation. Der Broker erhält Nachrichten von den Clients & sendet diese an andere Clients weiter, die in diesen bestimmten Topics subscribed haben.

1.1.2 MQTT-Client

Der MQTT-Client ist der Part, der innerhalb von z.B. IoT-Geräten eingebaut ist & für die Datenübertragung sowie Kommunikation verwendet wird. Der Client stellt eine Verbindung zum Broker her & subscribed zu ausgewählten Topics &/oder sendet ebenfalls auf diese Daten an den Broker zurück.

1.1.3 MQTT-Topic und dessen Grundregeln

Eine MQTT-Topic ist eine Art Channel für die Kommunikation. Diese können basierend auf Pfaden von einander getrennt werden. Dabei ermöglicht es einem, nur auf bestimmte Aktivitäten zu reagieren, anstatt die Nachrichten erst Clientseitig zu verwalten. Dies reduziert zum einen die Menge an Daten die versendet werden, ebenfalls benötigt der Client weniger Funktionalität bei der Nachrichtenpriorisierung & Zuordnung, da er nur das Erhält, was er selbst gewählt hat.

1.1.4 Topic Wildcard einstufig und mehrstufig

In MQTT gibt es zwei verschiedene Arten von Topic-Wildcards. Die erste Variante sind die einstufigen Wildcards, in MQTT wird hierfür das + benutzt. Eine Wildcard könnte daher wie folgt aussehen `abc/outlet01/+/audiQ3`. Die einstufige Wildcard würde nun alle Topics subscriben z.B. (`abc/outlet01/alert0/audiQ3`, `abc/outlet01/alert1/audiQ3` usw.).

Mehrstufige Topic Wildcards werden in MQTT mit dem # deklariert. Hierbei muss jedoch die Wildcard als letzter Pfad Parameter gesetzt sein. `abc/outlet01/alert0/#` wäre daher eine **valide** Wildcard. `abc/outlet01/#/audiQ3` ist hierbei eine **invalide** Wildcard

1.1.5 Stufenmodell Quality of Service (QoS)

Das Stufenmodell QoS ist für die Priorisierung von dem Datenverkehr im globalen & privaten Netzwerk zuständig. Hierbei gibt es folgende 4 Stufen der Priorisierung.

0-1 (Best Effort): Standarddatenverkehr (Web-Surfen, E-Mail).

2-3 (Best Effort+): Höhere Priorität als Standard.

4-5 (Echtzeit/Streaming): Reserviert für Video-Streaming, Telefonie (VoIP). Hier sind Verzögerungen (Jitter) und Paketverluste kritisch.

6-7 (Netzwerksteuerung): Höchste Priorität, reserviert für Netzwerk-Routing-Protokolle

1.1.6 Last Will Testament

Last Will Testament (LWT) ist für das Verwalten von Topics ausschlaggebend. Hierbei sendet der Broker bei einem unerwarteten disconnect von einem Client, eine Nachricht an alle anderen Clients in dieser Topic. In dem ersten Bild sehen Sie das Connect MQTT-Packet. Innerhalb dieses gibt es den Parameter

MQTT-Packet:	
CONNECT	
	
contains:	Example
clientId	"client-1"
cleanSession	true
username (optional)	"hans"
password (optional)	"letmein"
lastWillTopic (optional)	"/hans/will"
lastWillQos (optional)	2
lastWillMessage (optional)	"unexpected exit"
lastWillRetain (optional)	false
keepAlive	60

Abbildung 1.1: Last Will Message im Connect Packet (Grafik von HiveMQ)

MQTT-Packet:	
DISCONNECT	
	
no payload	

Abbildung 1.2: Last Will Message beim Disconnect (Grafik von HiveMQ)

lastWillMessage, hier kann eine eigene Nachricht verfasst werden. Ebenfalls gibt es weitere Einstellungsmöglichkeiten für den LWT, wie z.B. in welcher Topic der LWT gesendet werden soll, oder in welcher Priorität *QoS* die Nachricht versendet werden soll.

1.2 Verbindungsmanagement zwischen Client und Server

In den folgenden Sektionen, werden Ihnen folgende Begriffe erklärt. Dies geschieht innerhalb des Rahmens von *Skript Teil 3*.

1.2.1 Retained Messages

Eine retained Message ist eine Nachricht, die auf dem Server (in unserem Fall Broker) gespeicherte Nachricht. Damit diese Nachricht gespeichert wird, muss die **retained**-Flag aktiv sein. Dann speichert der Broker vorübergehend die Nachricht für diese Topic inkl. QoS auf dem Server. Sobald sich ein neuer Client mit der Topic per Subscribe verbindet, erhält er diese Nachricht.

1.2.2 Persistent Sessions

Persistent Sessions haben Ähnlichkeiten zu den Retained Messages, erweitern jedoch den Funktionsrahmen um ein weiteres. Bei einer Persistent Session speichert der Server alle nicht zustellbaren Nachrichten an den Client in den subscribten Topics & versucht es erneut, sobald der MQTT-Client wieder mit dem Broker verbunden ist. Damit der Broker überhaupt erstmal bescheid weiß, dass der Client eine Persistent Session erstellen möchte, muss dies bei dem Verbinden mit dem Broker, übermittelt werden.

1.2.3 Keep Alive

Keep Alive ist eine Funktionalität um eine Verbindung mit dem Broker aufrecht zu erhalten, obwohl kein Nachrichtenaustausch zwischen Broker & Client stattfindet. Diese Funktionalität kann ebenfalls bei dem erstellen des Clients eingestellt werden.

1.2.4 Web Sockets Transport

Web Sockets Transport ist eine Möglichkeit das MQTT Protokoll zu erweitern & für die Browser möglich zu machen. Dabei wird das MQTT Protokoll mit dem WebSocket Protokoll erweitert, dadurch können Browser ebenfalls an einer Topic im MQTT-Broker subscriben & die Daten verwalten.

1.3 Pub/Sub Messaging

Pub/Sub Messaging ist eine Protokollfamilie. Hierbei werden Nachrichten in sogenannten **Channels** oder **Topics** versendet. Man könnte dies mit den verschiedenen Bereichen in einem Amt vergleichen, denn je nach Channel, geht die Nachricht an andere Empfänger über. Jedoch bleiben für den Clienten die Empfänger unbekannt.

Pub/Sub ist ein serverbasiertes Nachrichtenprotokoll, d.h. die Nachricht, die ein Client versendet, werden an einen Server gesendet. Dieser *Verwaltet* die Eingehenden & Ausgehenden Verbindungen, den andere Clients können sich an diese Topics anklippen. Dadurch wird die Nachricht, sobald diese bei dem Server ankommt, automatisch an diese Clienten versendet. In dieser Grafik sieht man zwei verschiedene **Publisher** Clients. Diese senden eine Nachricht auf **/order** & **/inventory**. Beide senden die Nachricht mit dem Channel/Topic an den Pub/Sub Server. Hinter dem Pub/Sub Server befinden sich drei unterschiedliche Clients. Zwei dieser Clients hören auf Nachrichten innerhalb des **/order** Channels & einer auf den **/inventory**. Die Verbindung zwischen Client & Server bestehen aus WebSocket Verbindungen, damit der Server & der Client sich gegenseitig Nachrichten zusenden zu können.

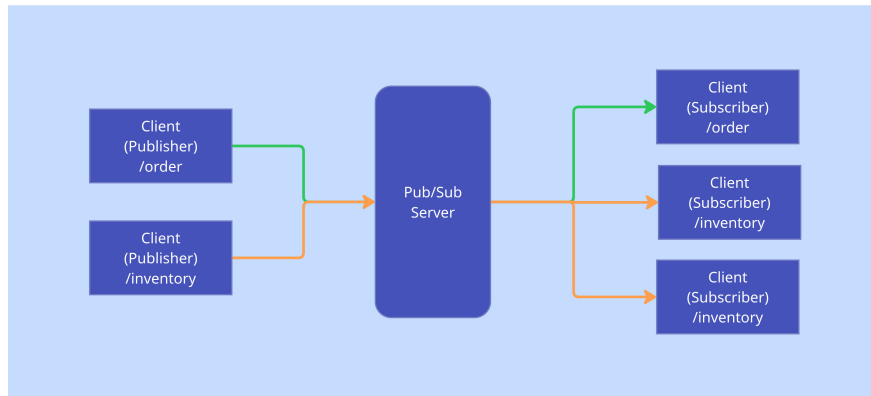


Abbildung 1.3: Grafische Darstellung des Pub/Sub Verfahrens

1.4 MQTT-Client

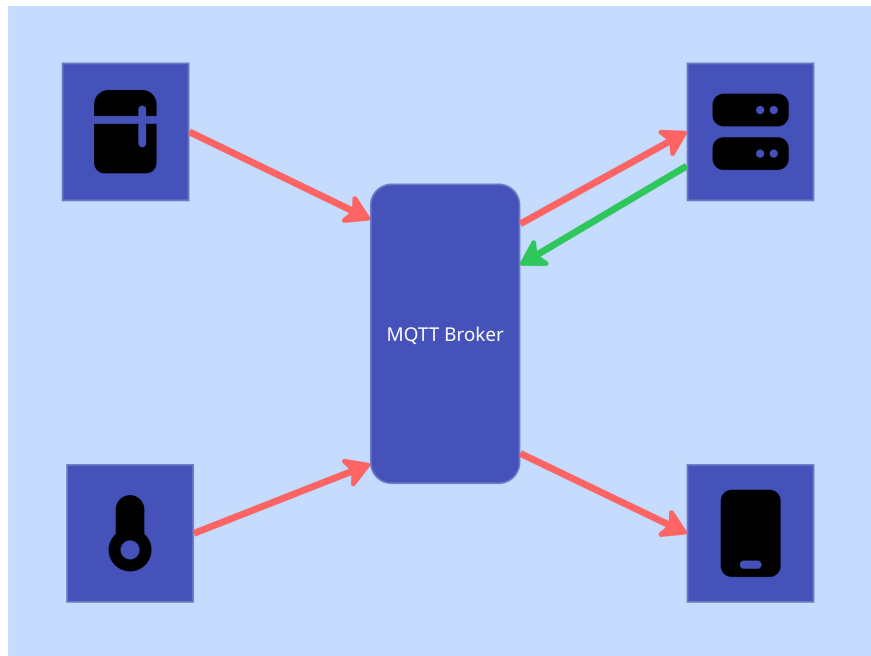


Abbildung 1.4: Grafik von MQTT

Diese Grafik verbildlicht die Kommunikation zwischen den einzelnen Geräten wie den Kühlschrank, Thermometer, Server & einem Handy. Hierbei werden die Daten mithilfe von dem MQTT-Client an den Broker gesendet. Der Broker sendet diese Daten dann an alle Clients, die den **Channel** subscribed haben.

Innerhalb dieses zweiten Teils werden Daten an einen externen Dienst versendet. Dafür gibt es verschiedene Wege, diesen Prozess zu absolvieren.

Das MQTT-Protokoll ist ein klassisches Pub/Sub Modell in der IoT Welt. Hierbei agiert es als leichtgewichtiges Messaging Protokoll für die Übermittlung von Daten, von unseren Client an den Server.

Kapitel 2

Internetverbindung für den ESP32

Damit Daten überhaupt von dem ESP32 an den MQTT-Broker übermittelt werden können, benötigt der ESP32 eine Internetverbindung.

Für die technische Umsetzung wurde sich auf die [ESP32 WLAN Modul](#) Dokumentation bezogen.

2.1 Planung

Der ESP32 soll sich mit einem W-LAN AccessPoint in der Nähe verbinden & dadurch eine internetfähige Verbindung aufbauen, damit später, eine Möglichkeit besteht, den MQTT-Broker zu erreichen.

2.2 Programmcode

```
1  import network
2
3  wlan = network.WLAN(network.STA_IF)
4
5  def do_connect():
6      wlan.active(True)
7      if not wlan.isconnected():
8          print('connecting to network...')
9          wlan.connect('FI24-Hotspot', 'BszWsw11#')
10         while not wlan.isconnected():
11             print("WLAN is not connected")
12             pass
13
14     print('network config:', wlan.ifconfig())
15
16 do_connect()
17
```

Zuerst importieren wir das Modul **network**. In Zeile 3 deklarieren wir das WLAN-Modul des ESP32, indem wir aus dem Netzwerk-Modul die Klasse **WLAN** instanziiieren. **network.STA_IF** bedeutet hierbei, das wir es als Station Interface deklarieren, also ein stationärer Zugriffspunkt (Access Point).

Damit wir später im Programmcode einfacher eine W-LAN Verbindung herstellen können, deklarieren wir die Funktion **do_connect()**. Innerhalb des Funktionsbodies, aktivieren wir zuerst die WLAN-Funktionalität, damit wir uns später mit einem Access Point verbinden können. Wir überprüfen zuerst, ob wir mit einem Access-Point verbunden sind. Dies verhindert, dass wir die Verbindung zu dem Access-Point neu aufbauen, obwohl wir bereits eine Verbindung haben.

In Zeile 9 verbinden wir uns mit dem definierten Access Point & überprüfen in der while-Schleife dann so lange, ob die Verbindung erfolgreich war. Erst nach einer erfolgreichen Verbindung lassen wir die while-Schleife los & geben die IP-Informationen in die Konsole aus.

Damit die WLAN Verbindung initialisiert werden kann, wird die Funktion `do_connect()` aufgerufen.

Kapitel 3

MQTT-Client

3.1 Planung

Der ESP32 soll bei einer internetfähigen Verbindung, sich mit dem MQTT-Broker verbinden & pro Durchlauf die Daten an den Broker senden, über eine zuvor definierte Topic.

3.2 Programmcode

```
1  from umqtt.simple import MQTTClient
2
3  client_id = "ESP_32_MQTT"
4  server = "mosquitto.nodered-fi.ipv64.net"
5  port = 0
6  user = "FI"
7  password = "FI"
8  keepalive = 1
9  mqtt_client = MQTTClient(client_id, server, port, user, password, keepalive=0,
10                           ssl=False, ssl_params={})
11
12 def mqtt_connect():
13     mqtt_client.connect()
14
15 def send_mqtt_message(message):
16     channel = "Met/FI/Timmel"
17     mqtt_client.publish(channel, message)
```

Kapitel 4

Verarbeitung der Daten in NODE-RED

4.1 MQTT-Client innerhalb von NODE-RED

Alle Daten kommen zuerst über einen MQTT-Client in Node-RED. Dieser Client ist an die selbe Topic subscribed, wie unser MQTT-Client für den ESP32 die Daten an den Broker sendet.

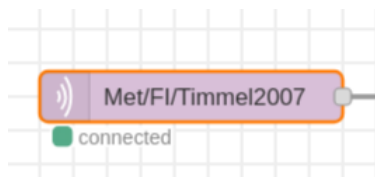


Abbildung 4.1:

Die Node *mqtt in* representiert einen MQTT Client, mitdem wir in diesem Beispiel eine Topic subscriben & die Daten anschließend innerhalb von Node-RED weiterverarbeiten.

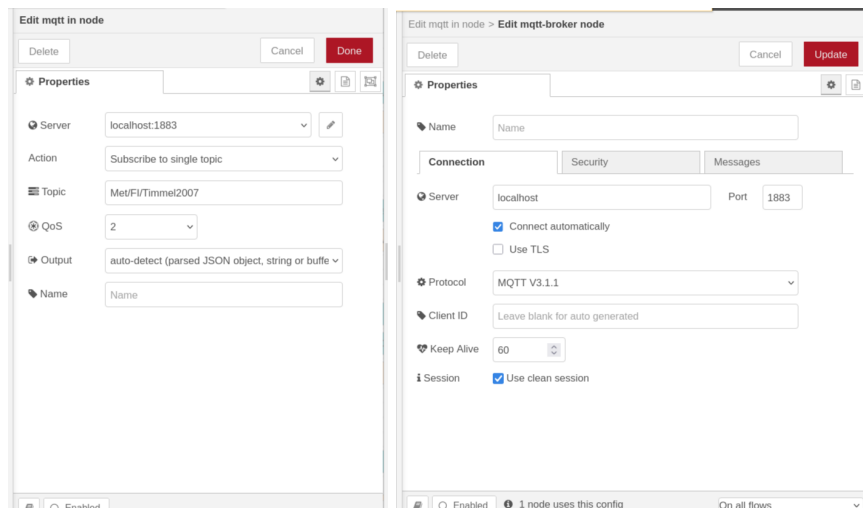


Abbildung 4.2:

Bevor die Node jedoch die Daten empfangen kann, müssen wir diese zuvor Einrichten. Hierfür setzen wir die richtige Serverkonfiguration ein, in unserem Fall die IP 127.0.0.1 mit dem dazugehörigen Port 1883. Ebenfalls müssen wir die Zugangsdaten für den MQTT-Broker übergeben, weshalb wir diese angeben

müssen. Nachdem wir die Serververbindung zum Broker eingerichtet haben, setzen wir die passende Topic zum subscriben. Bestätigen die Änderungen und deployen den Flow.

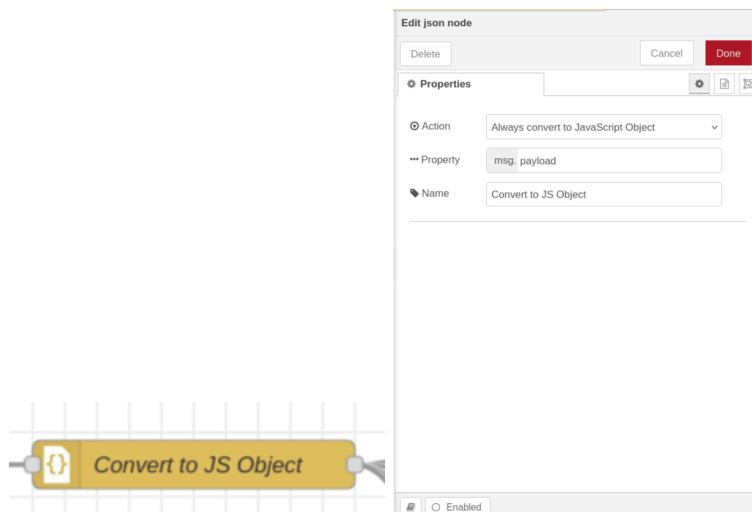


Abbildung 4.3:

Mit der **Convert to JSON-Object** Node, konvertieren wir den JSON-String in ein valides JSON-Objekt.

4.2 Temperatur

4.2.1 Vorbereitung der Daten für das UI



Abbildung 4.4:

Bevor wir die Daten für die Temperatur in die Visualisierung leiten können, müssen wir zuvor die richtigen Daten zuweisen.

```
1 msg.payload = msg.payload.Temperatur;
2
3 return msg;
4
```

Mithilfe dieses JavaScript Codes setzen wir die `msg.payload` Variable auf den Temperatur Wert, den wir zuvor erhalten haben.

4.2.2 UI Template für aktuelle Daten

```
1 <div class="temp-widget" ng-class="getTempClass(msg.payload)">
2   <div class="glass-overlay"></div>
3   <div class="content">
4     <span class="label">Temperatur</span>
5     <div class="temp-value">
6       <span class="number">{{msg.payload | number:1}}</span>
7       <span class="unit">°C</span>
8     </div>
```

```
9      <div class="status-indicator">
10        <div class="pulse-dot"></div>
11        <span class="status-text">{{getStatusText(msg.payload)}}</span>
12      </div>
13    </div>
14  </div>
15
```

4.2.3 UI Template für historie Daten

4.3 Luftfeuchtigkeit

4.3.1 Vorbereitung der Daten für das UI

4.3.2 UI Template für aktuelle Daten

4.3.3 UI Template für historie Daten

4.4 Bewegung

4.4.1 Vorbereitung der Daten für das UI

4.4.2 UI Template für aktuelle Daten

4.4.3 UI Template für historie Daten

4.5 Helligkeit

4.5.1 Vorbereitung der Daten für das UI

4.5.2 UI Template für aktuelle Daten

4.5.3 UI Template für historie Daten